Laboratório Prolog · Aula 13

Miguel Falcão Costacurta · RA 240113022 · Turma B
Gerado em 20/10/2026, 20:11:40 · tempo registrado no cronômetro: 22:03

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(scheme,funcional)
false.

Saída do ola.pl

```
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(java,A).
   paradigma(java,orientado_a_objetos)
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

um pergunta o que o prolog é o outro afirma que prolog é lógico.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

isso quer dizer que o programa não sabe isso, a única coisa que o programa sabe é que java é orientado a objetos e o prolog é lógico.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

7 respostas

4 pares

Saída do familia.pl

```
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
```

```
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

significa quem é mãe de mulher, ela teve 4 respostas.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

como o programa não tem a informação ele assume que ela é falsa, pode ser um problema quando esquecemos de adicionar as informações.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

A ordem em que os fatos mulher(...) aparecem no código-fonte. O Prolog procura as respostas de cima para baixo, seguindo a ordem dos fatos na base de conhecimento.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

false e false

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

f(X, X) = f(a, b) → falha.

f(X, X) = f(a, Y) → sucesso.

[A|B] = [] → falha.

2 + 3 = 5 → falha.

Saídas de regras.pl e unificacao.pl

```
--- avo: duas regras progenitor encadeadas
?- avo(_,_).
   avo(vern,jake)
   avo(vern,ana)
```

```
    avo(osvaldo,joana)
    avo(osvaldo,paulo)
    avo(osvaldo,bia)
    avo(osvaldo,jake)
    avo(osvaldo,ana)
    avo(osvaldo,leo)
?- avo(osvaldo,_).
    avo(osvaldo,joana)
    avo(osvaldo,paulo)
    avo(osvaldo,bia)
    avo(osvaldo,jake)
    avo(osvaldo,ana)
    avo(osvaldo,leo)
--- irmao, tio e primo
?- irmao(jake,_).
    false.
?- tio(_,jake).
    false.
?- primo(leo,_).
    false.
--- ancestral (recursiva)
?- ancestral(osvaldo,_).
    ancestral(osvaldo,vern)
    ancestral(osvaldo,tereza)
    ancestral(osvaldo,joana)
    ancestral(osvaldo,paulo)
    ancestral(osvaldo,jake)
    ancestral(osvaldo,ana)
    ancestral(osvaldo,leo)
    ancestral(osvaldo,bia)
?- ancestral(_,jake).
    ancestral(joana,jake)
    ancestral(jorge,jake)
    ancestral(vern,jake)
    ancestral(osvaldo,jake)
?- ancestral(jake,_).
    ancestral(jake,_)
    false.
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

A regra ancestral precisa de duas cláusulas porque é uma regra recursiva: uma cláusula define quando a relação é diretamente verdadeira e a outra permite encontrar ancestrais por meio de gerações intermediárias.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

A meta X \== Y impede que uma pessoa seja considerada irmã de si mesma.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

f(X, X) = f(a, b) não unifica porque X teria que ser igual a a e b ao mesmo tempo. Já f(X, X) = f(a, Y) unifica com X = a e Y = a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Sim. A regra avo permite consultar tanto os avós de Jake quanto os netos de Osvaldo. Isso é possível porque o Prolog usa variáveis e unificação para encontrar valores que satisfaçam a relação, enquanto funções tradicionais em Java ou Scheme normalmente executam operações a partir dos argumentos fornecidos.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

X = [], Y = [1, 2, 3]

X = [1], Y = [2, 3]

X = [1, 2], Y = [3]

X = [1, 2, 3], Y = []

Saída do listas.pl

```
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
   membro(b,[a,b,c])
?- membro(z,[a,b,c]).
   false.
?- membro(_,[a,b,c]).
   membro(a,[a,b,c])
   membro(b,[a,b,c])
   membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],_).
   tamanho([a,b,c],3)
?- tamanho([],_).
   tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],_).
   concatena([1,2],[3,4],[1,2,3,4])
?- concatena(_,[3],[1,2,3]).
   concatena([1,2],[3],[1,2,3])
?- concatena([1],_,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(_,_,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],_).
   inverte([1,2,3],[3,2,1])
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

O caso base é tamanho([], 0), que determina que uma lista vazia tem tamanho zero. O passo recursivo é tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1. A cada chamada, o Prolog precisa calcular o tamanho da cauda e somar 1 ao resultado, até chegar à lista vazia.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

concatena(X, Y, [1, 2, 3]) tem quatro respostas porque a lista pode ser dividida em duas partes de quatro maneiras: [] e [1,2,3]; [1] e [2,3]; [1,2] e [3]; [1,2,3] e []. O append do Scheme normalmente recebe as listas de entrada e calcula uma lista resultante; ele não busca automaticamente todas as divisões possíveis de uma lista dada.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O mecanismo é a unificação combinada com retrocesso (backtracking). O Prolog encontra uma solução, retorna para tentar outras possibilidades e gera a, b e c como valores de X, sem precisar de um laço explícito.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3: não calcula; X recebe a estrutura 2+3.

X is 2 + 3: calcula a expressão e X recebe 5.

fatorial(N, 120): dá erro de instanciação, porque N está livre e a regra tenta fazer operações aritméticas com uma variável cujo valor ainda não é conhecido.

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- _ = 2+3.
   2+3 = 2+3
?- _ is 2+3.
   5 is 2+3
?- 5 =:= 2+3.
   5 =:= 2+3
?- 2+3 = 5.
   false.
?- _ is 7 mod 3.
   1 is 7 mod 3
?- _ is 7//2.
   3 is 7//2
?- _ is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,_).
   fatorial(5,120)
?- fatorial(0,_).
   fatorial(0,1)
?- fib(10,_).
   fib(10,55)
?- soma([1,2,3,4],_).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
   ERRO: error(instantiation_error,...)
   ERRO: error(instantiation_error,...)
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= faz unificação, sem calcular: X = 2 + 3 faz X valer a estrutura 2+3.

is calcula a expressão: X is 2 + 3 resulta em X = 5.

=:= compara valores numéricos: 5 =:= 2 + 3 retorna true.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

fatorial(N, 120) dá erro porque N está livre e a regra usa operações aritméticas que exigem seu valor conhecido. Já concatena(X, [3], [1, 2, 3]) funciona porque a unificação permite descobrir X e relacionar as listas sem precisar calcular valores numéricos.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Ambos usam recursão e um caso base (fatorial(0, 1) no Prolog e fact 0 = 1 no Haskell). A diferença é que, no Prolog, o resultado é passado como um argumento extra, enquanto no Haskell a função retorna diretamente o resultado.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro1(X): não dá false; encontra Osvaldo, Paulo e Jake como solteiros.

solteiro2(X): dá false, pois tenta verificar \+ casado(X) com X ainda livre.

maximo2(5, 3, M): dá duas respostas, M = 5 e M = 3, porque a segunda regra não tem teste.

maximo3(5, 3, 3): dá true, pois a segunda regra aceita M = 3 depois que a primeira cláusula falha na unificação com o terceiro argumento.

Saída de controle.pl e mensagem de ancestral_ruim

```
ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)

Fatal Error: local stack overflow (size: 16384 Kb, reached: 16383 Kb, environment variable
used: LOCALSZ)
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores: 24 colorações.

Com 2 cores: 0 colorações, porque PR, SP e MS fazem fronteira entre si e precisam de três cores diferentes.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
   false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog encontrou duas respostas de mesmo_gosto(X, Y): (ana, caio) e (caio, ana). Quando tentou (ana, ana), a meta X \== Y falhou porque as duas variáveis representavam a mesma pessoa. O Prolog voltou ao ponto de escolha

e tentou outras combinações.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

solteiro1(X) gera primeiro os homens e depois verifica se não são casados. Por isso, encontra Osvaldo, Paulo e Jake. solteiro2(X) dá false porque tenta verificar \+ casado(X) com X ainda livre; como existem pessoas casadas, a negação falha. Osvaldo é considerado solteiro porque a base de fatos não informa que ele é casado. Ser pai não significa necessariamente ser casado.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

maximo3(5, 3, 3) dá true porque o corte impede tentar a segunda cláusula, mas a primeira cláusula não exige que o terceiro argumento seja igual a X antes do corte. Para corrigir, faça a unificação antes do corte:

prolog
maximo3(X, Y, X) :- X >= Y, !.
maximo3(_, Y, Y).
Uma alternativa mais segura é:

prolog
maximo3(X, Y, M) :- X >= Y, !, M = X.
maximo3(_, Y, Y).
Correção da 5.3: a primeira cláusula não chega ao corte em maximo3(5, 3, 3), pois a unificação dos argumentos falha antes de executar o corpo. A segunda cláusula então retorna true. Para corrigir, use:

prolog
maximo3(X, Y, M) :- X >= Y, !, M = X.
maximo3(_, Y, Y).

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim estoura a pilha porque chama a si mesma antes de verificar a relação progenitor, entrando em recursão infinita. ancestral_bom primeiro verifica progenitor(X, Z) e só depois faz a chamada recursiva, avançando pela árvore familiar até encontrar uma solução.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

3 cores: 24 colorações.

2 cores: 0 colorações, porque PR, SP e MS formam um triângulo e precisam de três cores diferentes.

4 cores: 1728 colorações.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No paradigma lógico, descrevemos relações: concatena(X, Y, [1,2,3]) encontra todas as divisões possíveis da lista. No funcional, como em anexar no Scheme, aplicamos uma função para obter um resultado.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

Duas deficiências do Prolog são a dependência da ordem das metas, vista em solteiro1 e solteiro2, e o risco de recursão infinita, visto em ancestral_ruim, que estoura a pilha.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas de regras e relações, como sistemas especialistas e consultas sobre parentesco, pois ele encontra soluções automaticamente. Não escolheria para aplicações que exigem controle detalhado do estado, interfaces gráficas ou processamento intensivo, para as quais Java costuma ser mais adequado.